XTI302-CYB - Administration Systèmes avancées et Scripting

Compte rendu du TP Socle - Séance 3

Droits fins, délégation et accès SSH sécurisé

Nom / Prénom : .....

Groupe : ..... Date : .....

Mise en place

Environnement utilisé

J'ai fait le TP sur une machine Ubuntu 24.04 (famille Debian) dans laquelle j'ai les droits sudo. Le nom de la machine est `tp-socle` et mon compte s'appelle `etu` (membre du groupe sudo).

Je n'avais pas de poste séparé capable de joindre la machine en SSH, j'ai donc suivi le Plan B : un compte local `poste` joue le rôle de mon poste de travail, et plusieurs terminaux sont ouverts en parallèle.

Point particulier : la machine ne démarre pas avec systemd (PID 1 n'est pas systemd). Les commandes `systemctl` et `journalctl` répondent donc « System has not been booted with systemd » ou « No journal files were found ». J'ai utilisé à la place `service ssh status` / `service ssh reload`, et les journaux de sudo et de sshd sont lus dans `/var/log/auth.log` (rsyslog), qui contient exactement les mêmes messages.

```
etu@tp-socle:~$ id
uid=30033(etu) gid=30033(etu) groups=30033(etu),27(sudo)
etu@tp-socle:~$
```

Figure 1 : mon compte etu est bien dans le groupe sudo

```
etu@tp-socle:~$ sudo apt install -y acl openssh-server
[sudo] password for etu:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
acl is already the newest version (2.3.2-1build1.1).
openssh-server is already the newest version (1:9.6p1-3ubuntu13.19).
0 upgraded, 0 newly installed, 0 to remove and 33 not upgraded.
etu@tp-socle:~$ dpkg -l acl openssh-server | tail -2
ii  acl            2.3.2-1build1.1      amd64        access control list - utilities
ii  openssh-server 1:9.6p1-3ubuntu13.19 amd64        secure shell (SSH) server, for secure access fr
om remote machines
etu@tp-socle:~$
```

Figure 2 : installation des paquets acl et openssh-server

```
etu@tp-socle:~$ systemctl status ssh --no-pager
System has not been booted with systemd as init system (PID 1). Can't operate.
Failed to connect to bus: Host is down
etu@tp-socle:~$ sudo service ssh status
 * sshd is running
etu@tp-socle:~$
```

Figure 3 : systemctl indisponible, état du service vérifié avec service

Création des comptes de test

```
etu@tp-socle:~$ sudo useradd -m -s /bin/bash poste
etu@tp-socle:~$ sudo groupadd projet
etu@tp-socle:~$ for U in alice bob carole; do
>   sudo useradd -m -s /bin/bash "$U"
>   echo "$U:Tp-Seance3-2026" | sudo chpasswd
> done
etu@tp-socle:~$ sudo usermod -aG projet alice
etu@tp-socle:~$ sudo usermod -aG projet carole
etu@tp-socle:~$
```

Figure 4 : création du compte poste (Plan B), du groupe projet et des comptes alice, bob, carole

Exercice 0 - Vérifier l'environnement

```
etu@tp-socle:~$ id alice; id bob; id carole
uid=30035(alice) gid=30036(alice) groups=30036(alice),30035(projet)
uid=30036(bob) gid=30037(bob) groups=30037(bob)
uid=30037(carole) gid=30038(carole) groups=30038(carole),30035(projet)
etu@tp-socle:~$ getent group projet
projet:x:30035:alice,carole
etu@tp-socle:~$ getfacl --version
getfacl 2.3.2
etu@tp-socle:~$ sudo service ssh status
 * sshd is running
etu@tp-socle:~$
```

Figure 5 : vérification des comptes, du groupe projet, des outils ACL et du service SSH

Vérification	Résultat obtenu
`id bob`	uid=30036(bob) gid=30037(bob) groups=30037(bob) : bob n'est pas dans projet
`getent group projet`	projet:x:30035:alice,carole
`getfacl --version`	getfacl 2.3.2
État du service ssh	sshd is running (vérifié avec `service ssh status`)

Exercice 1 - Constater les limites de rwx

```
etu@tp-socle:~$ sudo mkdir -p /srv/partage
[sudo] password for etu:
etu@tp-socle:~$ sudo chown alice:projet /srv/partage
etu@tp-socle:~$ sudo chmod 750 /srv/partage
etu@tp-socle:~$ sudo -u alice sh -c 'echo "Budget 2026 : version de travail" > /srv/partage/rapport.
txt'
etu@tp-socle:~$ sudo -u alice chgrp projet /srv/partage/rapport.txt
etu@tp-socle:~$ sudo -u alice chmod 640 /srv/partage/rapport.txt
etu@tp-socle:~$ ls -ld /srv/partage
drwxr-x--- 2 alice projet 4096 Oct  7 14:05 /srv/partage
etu@tp-socle:~$ ls -l /srv/partage
ls: cannot open directory '/srv/partage': Permission denied
etu@tp-socle:~$ sudo ls -l /srv/partage
total 4
-rw-r----- 1 alice projet 33 Oct  7 14:05 rapport.txt
etu@tp-socle:~$
```

Figure 6 : mise en place du dossier partagé

Remarque : `ls -l /srv/partage` lancé avec mon compte etu est refusé, ce qui est normal puisque etu n'est ni alice ni membre de projet (droits 750). J'ai donc utilisé `sudo ls -l` pour afficher le contenu.

```
etu@tp-socle:~$ sudo -u carole cat /srv/partage/rapport.txt
Budget 2026 : version de travail
etu@tp-socle:~$ sudo -u bob cat /srv/partage/rapport.txt
cat: /srv/partage/rapport.txt: Permission denied
etu@tp-socle:~$ sudo -u bob ls /srv/partage
ls: cannot open directory '/srv/partage': Permission denied
etu@tp-socle:~$
```

Figure 7 : tests de lecture par carole et bob

Commande	Prédiction	Résultat
`sudo -u carole cat ...`	réussit	réussit : affiche « Budget 2026 : version de travail »
`sudo -u bob cat ...`	échoue	échoue : Permission denied
`sudo -u bob ls ...`	échoue	échoue : cannot open directory, Permission denied

Réponse à la question 2 :

bob est bloqué par les droits du répertoire. `ls -ld` affiche `drwxr-x--- alice projet` : bob n'est ni le propriétaire ni dans le groupe projet, il tombe donc dans la catégorie « autres » qui a `---`. Sans le droit x il ne peut pas traverser `/srv/partage`, donc il ne peut atteindre aucun fichier dedans, même si le fichier était lisible. (Le fichier en 640 le bloquerait aussi, mais le répertoire bloque en premier.)

Question 3 : solutions avec les droits classiques

Solution avec les droits classiques	Inconvénient
Ajouter bob au groupe projet (`usermod -aG projet bob`)	bob obtient tous les droits du groupe sur tout ce qui appartient à projet, pas seulement la lecture du rapport (et les futures écritures si le groupe a w)
Ouvrir aux autres : `chmod o+x /srv/partage` et `chmod o+r rapport.txt`	tous les comptes de la machine peuvent lire le rapport, pas seulement bob
Donner le fichier à bob : `chown bob rapport.txt` (et o+x sur le dossier)	bob devient propriétaire : il peut modifier le fichier et changer ses droits, alice perd la main dessus

Exercice 2 - Poser et lire une ACL

```
etu@tp-socle:~$ sudo -u alice setfacl -m u:bob:r /srv/partage/rapport.txt
etu@tp-socle:~$ sudo ls -l /srv/partage
total 4
-rw-r-----+ 1 alice projet 33 Oct  7 14:05 rapport.txt
etu@tp-socle:~$ sudo getfacl /srv/partage/rapport.txt
getfacl: Removing leading '/' from absolute path names
# file: srv/partage/rapport.txt
# owner: alice
# group: projet
user::rw-
user:bob:r--
group::r--
mask::r--
other::---

etu@tp-socle:~$
```

Figure 8 : ACL u:bob:r posée par alice, ls -l et getfacl

Étape 2 : un + est apparu à la fin des droits (`-rw-r-----+`). Il indique que le fichier porte des ACL.

Étape 3 : sortie de getfacl après l'étape 1 et explication :

Ligne	Signification
`# file / # owner / # group`	nom du fichier, propriétaire (alice) et groupe propriétaire (projet)
`user::rw-`	droits du propriétaire alice : lecture et écriture
`user:bob:r--`	entrée nommée ajoutée pour bob : lecture seule
`group::r--`	droits du groupe propriétaire projet : lecture
`mask::r--`	masque : plafond des droits pour les entrées nommées et le groupe (calculé automatiquement)
`other::---`	les autres n'ont aucun droit

```
etu@tp-socle:~$ sudo -u bob cat /srv/partage/rapport.txt
cat: /srv/partage/rapport.txt: Permission denied
etu@tp-socle:~$
```

Figure 9 : étape 4 : bob ne peut toujours pas lire

Étape 4 : bob est encore refusé. L'ACL du fichier l'autorise bien, mais il ne peut pas traverser le répertoire `/srv/partage` (other `---`), donc il n'arrive jamais jusqu'au fichier.

```
etu@tp-socle:~$ sudo -u alice setfacl -m u:bob:x /srv/partage
etu@tp-socle:~$ sudo -u bob cat /srv/partage/rapport.txt
Budget 2026 : version de travail
etu@tp-socle:~$ sudo -u bob ls /srv/partage
ls: cannot open directory '/srv/partage': Permission denied
etu@tp-socle:~$
```

Figure 10 : étape 5 : droit x seul sur le dossier, bob lit mais ne liste pas

```
etu@tp-socle:~$ sudo -u alice setfacl -m u:carole:rw /srv/partage/rapport.txt
etu@tp-socle:~$ sudo -u carole sh -c 'echo "Ajout de carole" >> /srv/partage/rapport.txt'
etu@tp-socle:~$ sudo -u bob sh -c 'echo "Ajout de bob" >> /srv/partage/rapport.txt'
sh: 1: cannot create /srv/partage/rapport.txt: Permission denied
etu@tp-socle:~$ sudo cat /srv/partage/rapport.txt
Budget 2026 : version de travail
Ajout de carole
etu@tp-socle:~$
```

Figure 11 : étape 6 : carole peut écrire, bob non

```
etu@tp-socle:~$ sudo -u alice setfacl -x u:bob /srv/partage/rapport.txt
etu@tp-socle:~$ sudo getfacl /srv/partage/rapport.txt
getfacl: Removing leading '/' from absolute path names
# file: srv/partage/rapport.txt
# owner: alice
# group: projet
user::rw-
user:carole:rw-
group::r--
mask::rw-
other::---

etu@tp-socle:~$ sudo -u bob cat /srv/partage/rapport.txt
cat: /srv/partage/rapport.txt: Permission denied
etu@tp-socle:~$
```

Figure 12 : étape 7 : entrée de bob retirée, il ne lit plus le rapport

Test	**Prédiction**	**Résultat**
Étape 4 : bob lit	échoue	échoue (Permission denied)
Étape 5 : bob lit	réussit	réussit
Étape 5 : bob liste	échoue	échoue (cannot open directory)
Étape 6 : carole écrit	réussit	réussit
Étape 6 : bob écrit	échoue	échoue (cannot create, Permission denied)
Étape 7 : bob lit	échoue	échoue (Permission denied)

Commande de l'étape 6 : `sudo -u alice setfacl -m u:carole:rw /srv/partage/rapport.txt`

Commande de l'étape 7 : `sudo -u alice setfacl -x u:bob /srv/partage/rapport.txt`

Réponse à la question 8 :

Sur un répertoire, `x` permet de traverser (accéder à une entrée dont on connaît le nom) et `r` permet de lister les noms. bob a `x` sans `r` : il peut ouvrir un fichier s'il connaît son nom et si le fichier l'autorise, mais il ne peut pas voir ce que contient le dossier. Ce n'est pas une vraie protection : cacher les noms ne suffit pas, un nom peut se deviner (rapport.txt, budget…). La protection doit venir des droits du fichier lui-même. D'ailleurs à l'étape 7 l'entrée `u:bob:x` reste sur le dossier : il faudrait aussi la retirer à la fin de la mission.

Exercice 3 - Le masque, plafond des droits

```
etu@tp-socle:~$ sudo getfacl /srv/partage/rapport.txt
getfacl: Removing leading '/' from absolute path names
# file: srv/partage/rapport.txt
# owner: alice
# group: projet
user::rw-
user:carole:rw-
group::r--
mask::rw-
other::---

etu@tp-socle:~$
```

Figure 13 : étape 1 : masque rw- avec l'entrée de carole

```
etu@tp-socle:~$ sudo -u alice chmod 640 /srv/partage/rapport.txt
etu@tp-socle:~$ sudo getfacl /srv/partage/rapport.txt
getfacl: Removing leading '/' from absolute path names
# file: srv/partage/rapport.txt
# owner: alice
# group: projet
user::rw-
user:carole:rw-                 #effective:r--
group::r--
mask::r--
other::---

etu@tp-socle:~$ sudo -u carole sh -c 'echo "Test carole" >> /srv/partage/rapport.txt'
sh: 1: cannot create /srv/partage/rapport.txt: Permission denied
etu@tp-socle:~$ sudo ls -l /srv/partage/rapport.txt
-rw-r-----+ 1 alice projet 49 Oct  7 14:05 /srv/partage/rapport.txt
etu@tp-socle:~$
```

Figure 14 : étapes 2 à 4 : chmod 640 réduit le masque, carole ne peut plus écrire

Étape 3 : c'est la ligne `mask::` qui a changé (rw- devient r--). La ligne `user:carole:rw-` est toujours là, mais getfacl ajoute `#effective:r--` : ce commentaire indique les droits réellement appliqués, une fois le masque pris en compte. carole n'a donc plus que la lecture.

Étape 4 : prédiction : l'écriture de carole échoue. Résultat : échec, « cannot create … Permission denied ».

```
etu@tp-socle:~$ sudo -u alice setfacl -m m::rw /srv/partage/rapport.txt
etu@tp-socle:~$ sudo getfacl /srv/partage/rapport.txt
getfacl: Removing leading '/' from absolute path names
# file: srv/partage/rapport.txt
# owner: alice
# group: projet
user::rw-
user:carole:rw-
group::r--
mask::rw-
other::---

etu@tp-socle:~$ sudo -u carole sh -c 'echo "Test carole apres masque" >> /srv/partage/rapport.txt'
etu@tp-socle:~$ sudo ls -l /srv/partage/rapport.txt
-rw-rw----+ 1 alice projet 74 Oct  7 14:05 /srv/partage/rapport.txt
etu@tp-socle:~$
```

Figure 15 : étape 6 : masque remis à rw, carole écrit de nouveau

Moment	Ligne mask::	Ligne user:carole
Étape 1	`mask::rw-`	`user:carole:rw-`
Étape 3	`mask::r--`	`user:carole:rw-` `#effective:r--`
Étape 6	`mask::rw-`	`user:carole:rw-`

Réponse à la question 5 :

Quand un fichier porte des ACL, la partie « groupe » de `ls -l` n'affiche plus les droits du groupe propriétaire mais la valeur du masque : `-rw-r-----+` quand le masque vaut r--, `-rw-rw----+` après `setfacl -m m::rw`, alors que `group::` est resté `r--`. Et `chmod g=...` modifie le masque, pas `group::`.

Réponse à la question 7 :

Le masque permet de limiter d'un coup tous les droits accordés par les ACL (utilisateurs nommés, groupes nommés et groupe propriétaire) sans supprimer les entrées une par une, par exemple pour geler temporairement les écritures sur un dossier pendant un audit, puis de tout rétablir en remettant le masque. Il faut aussi savoir qu'un simple chmod le modifie, ce qui peut casser des ACL sans qu'on s'en rende compte.

Exercice 4 - Héritage : les ACL par défaut

```
etu@tp-socle:~$ sudo mkdir /srv/partage/commun
etu@tp-socle:~$ sudo chown alice:projet /srv/partage/commun
etu@tp-socle:~$ sudo -u alice sh -c 'echo "ancien contenu" > /srv/partage/commun/ancien.txt'
etu@tp-socle:~$ sudo -u alice setfacl -m g:projet:rwX /srv/partage/commun
etu@tp-socle:~$ sudo -u alice setfacl -d -m g:projet:rwX /srv/partage/commun
etu@tp-socle:~$ sudo getfacl /srv/partage/commun
getfacl: Removing leading '/' from absolute path names
# file: srv/partage/commun
# owner: alice
# group: projet
user::rwx
group::r-x
group:projet:rwx
mask::rwx
other::r-x
default:user::rwx
default:group::r-x
default:group:projet:rwx
default:mask::rwx
default:other::r-x

etu@tp-socle:~$
```

Figure 16 : ACL d'accès et ACL par défaut posées sur commun

Les lignes qui commencent par `default:` sont les ACL par défaut : elles ne donnent aucun droit sur le dossier lui-même, elles seront copiées sur ce qui sera créé dedans.

```
etu@tp-socle:~$ sudo -u alice sh -c 'echo "nouveau contenu" > /srv/partage/commun/nouveau.txt'
etu@tp-socle:~$ sudo getfacl /srv/partage/commun/ancien.txt /srv/partage/commun/nouveau.txt
getfacl: Removing leading '/' from absolute path names
# file: srv/partage/commun/ancien.txt
# owner: alice
# group: alice
user::rw-
group::rw-
other::r--

# file: srv/partage/commun/nouveau.txt
# owner: alice
# group: alice
user::rw-
group::r-x                      #effective:r--
group:projet:rwx                #effective:rw-
mask::rw-
other::r--

etu@tp-socle:~$
```

Figure 17 : comparaison de ancien.txt et nouveau.txt

Étape 3 : `ancien.txt` (créé avant) n'a aucune ACL. `nouveau.txt` a hérité de `group:projet:rwx`, limité à rw- par le masque (le x n'est pas donné à un fichier créé avec des droits 666).

```
etu@tp-socle:~$ sudo -u carole sh -c 'echo "carole" >> /srv/partage/commun/ancien.txt'
sh: 1: cannot create /srv/partage/commun/ancien.txt: Permission denied
etu@tp-socle:~$ sudo -u carole sh -c 'echo "carole" >> /srv/partage/commun/nouveau.txt'
etu@tp-socle:~$
```

Figure 18 : étape 4 : carole n'écrit que dans nouveau.txt

```
etu@tp-socle:~$ sudo -u alice setfacl -R -m g:projet:rwX /srv/partage/commun
etu@tp-socle:~$ sudo -u carole sh -c 'echo "carole" >> /srv/partage/commun/ancien.txt'
etu@tp-socle:~$ sudo getfacl /srv/partage/commun/ancien.txt
getfacl: Removing leading '/' from absolute path names
# file: srv/partage/commun/ancien.txt
# owner: alice
# group: alice
user::rw-
group::rw-
group:projet:rw-
mask::rw-
other::r--

etu@tp-socle:~$
```

Figure 19 : étape 5 : setfacl -R, carole peut écrire dans ancien.txt

```
etu@tp-socle:~$ sudo -u carole mkdir /srv/partage/commun/archives
etu@tp-socle:~$ sudo getfacl /srv/partage/commun/archives
getfacl: Removing leading '/' from absolute path names
# file: srv/partage/commun/archives
# owner: carole
# group: carole
user::rwx
group::r-x
group:projet:rwx
mask::rwx
other::r-x
default:user::rwx
default:group::r-x
default:group:projet:rwx
default:mask::rwx
default:other::r-x

etu@tp-socle:~$
```

Figure 20 : étape 6 : le sous-dossier archives a hérité des ACL

Étape 6 : oui, `archives` a hérité à la fois de l'ACL d'accès `group:projet:rwx` et des lignes `default:`, donc l'héritage continue dans les sous-dossiers.

```
etu@tp-socle:~$ sudo -u alice sh -c 'echo "prepare chez moi" > /home/alice/deplace.txt'
etu@tp-socle:~$ sudo -u alice mv /home/alice/deplace.txt /srv/partage/commun/
etu@tp-socle:~$ sudo getfacl /srv/partage/commun/deplace.txt
getfacl: Removing leading '/' from absolute path names
# file: srv/partage/commun/deplace.txt
# owner: alice
# group: alice
user::rw-
group::rw-
other::r--

etu@tp-socle:~$ sudo -u carole sh -c 'echo "carole" >> /srv/partage/commun/deplace.txt'
sh: 1: cannot create /srv/partage/commun/deplace.txt: Permission denied
etu@tp-socle:~$ sudo ls -l /srv/partage/commun
total 16
-rw-rw-r--+ 1 alice  alice    22 Oct  7 14:05 ancien.txt
drwxrwxr-x+ 2 carole carole 4096 Oct  7 14:05 archives
-rw-rw-r--  1 alice  alice    17 Oct  7 14:05 deplace.txt
-rw-rwxr--+ 1 alice  alice    23 Oct  7 14:05 nouveau.txt
etu@tp-socle:~$
```

Figure 21 : étape 7 : le fichier déplacé avec mv n'a pas l'ACL du groupe projet

Étape 7 : `deplace.txt` ne porte pas l'ACL du groupe projet et carole ne peut pas le modifier. `mv` sur le même système de fichiers ne crée pas de nouveau fichier, il garde l'inode et ses droits d'origine.

Fichier	ACL group:projet présente ?	carole peut écrire ?
ancien.txt (avant -R)	non	non
nouveau.txt	oui (effective rw-)	oui
ancien.txt (après -R)	oui	oui
deplace.txt	non	non

Commande de l'étape 5 : `sudo -u alice setfacl -R -m g:projet:rwX /srv/partage/commun`

Réponse à la question 8 :

Une ACL par défaut n'agit qu'au moment de la création d'un fichier. Elle ne touche pas les fichiers déjà présents (il faut `-R`), ni ceux qui arrivent par `mv` (ils gardent leurs droits), ni ceux dont on change ensuite les droits avec chmod (masque réduit). Il faut donc combiner ACL par défaut, application récursive sur l'existant et vérification régulière (getfacl ou un script).

Exercice 5 - Déléguer un droit avec sudo

```
etu@tp-socle:~$ sudo -l
[sudo] password for etu:
Matching Defaults entries for etu on tp-socle:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User etu may run the following commands on tp-socle:
    (ALL : ALL) ALL
etu@tp-socle:~$
```

Figure 22 : étape 1 : mes droits sudo

Étape 1 : mon compte peut tout faire en tant que n'importe qui : `(ALL : ALL) ALL`, grâce au groupe sudo.

```
etu@tp-socle:~$ sudo -iu bob
bob@tp-socle:~$ sudo -l
[sudo] password for bob:
Sorry, user bob may not run sudo on tp-socle.
bob@tp-socle:~$ exit
logout
etu@tp-socle:~$
```

Figure 23 : étape 2 : sudo -l dans un shell de bob

Étape 2 : sudo demande le mot de passe de bob puis répond « Sorry, user bob may not run sudo on tp-socle. »

Étape 3 : `command -v systemctl` donne `/usr/bin/systemctl`, puis création de la règle avec `sudo EDITOR=nano visudo -f /etc/sudoers.d/tp-operateurs` :

```
  GNU nano 7.2                    /etc/sudoers.d/tp-operateurs.tmp *
# Seance 3 - bob peut redemarrer cron, et rien d'autre
bob ALL=(root) /usr/bin/systemctl restart cron


^G Help       ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit       ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

Figure 24 : contenu saisi dans visudo

```
etu@tp-socle:~$ sudo cat /etc/sudoers.d/tp-operateurs
# Seance 3 - bob peut redemarrer cron, et rien d'autre
bob ALL=(root) /usr/bin/systemctl restart cron

etu@tp-socle:~$ ls -l /etc/sudoers.d/
total 8
-r--r----- 1 root root 1068 Jan 29  2024 README
-rw-r----- 1 root root  103 Oct  7 14:12 tp-operateurs
etu@tp-socle:~$
```

Figure 25 : étape 4 : droits du fichier créé

Étape 4 : le fichier appartient à root:root avec les droits `-rw-r-----` (640). Plus loin, `visudo -c` signale « bad permissions, should be mode 0440 », j'ai donc corrigé avec `sudo chmod 440` (voir la version finale).

```
etu@tp-socle:~$ sudo -iu bob
bob@tp-socle:~$ sudo -l
[sudo] password for bob:
Matching Defaults entries for bob on tp-socle:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User bob may run the following commands on tp-socle:
    (root) /usr/bin/systemctl restart cron
bob@tp-socle:~$ sudo systemctl restart cron
System has not been booted with systemd as init system (PID 1). Can't operate.
Failed to connect to bus: Host is down
bob@tp-socle:~$ sudo systemctl stop cron
Sorry, user bob is not allowed to execute '/usr/bin/systemctl stop cron' as root on tp-socle.
bob@tp-socle:~$ sudo systemctl restart ssh
Sorry, user bob is not allowed to execute '/usr/bin/systemctl restart ssh' as root on tp-socle.
bob@tp-socle:~$ exit
logout
etu@tp-socle:~$
```

Figure 26 : étape 5 : tests dans un shell de bob

Étape 5 : sudo demande le mot de passe de bob (et pas celui de root). La commande `restart cron` est acceptée par sudo : il n'y a pas de message « not allowed » et elle apparaît dans le journal avec `COMMAND=`. Le message affiché ensuite vient de systemctl lui-même, parce que la machine n'a pas systemd : l'autorisation sudo fonctionne. `stop cron` et `restart ssh` sont refusées par sudo.

Commande de bob	**Prédiction**	**Résultat**
`sudo systemctl restart cron`	autorisée	autorisée par sudo (journal : COMMAND=/usr/bin/systemctl restart cron)
`sudo systemctl stop cron`	refusée	refusée : « Sorry, user bob is not allowed to execute … »
`sudo systemctl restart ssh`	refusée	refusée : « Sorry, user bob is not allowed to execute … »

```
etu@tp-socle:~$ sudo journalctl -t sudo -n 10 --no-pager
No journal files were found.
-- No entries --
etu@tp-socle:~$ sudo grep 'sudo:.*bob' /var/log/auth.log | tail -n 6
2026-10-07T14:12:08.468651+00:00 tp-socle sudo: pam_unix(sudo-i:session): session opened for user bo
b(uid=30036) by (uid=30033)
2026-10-07T14:12:08.738750+00:00 tp-socle sudo:      bob : TTY=pts/1 ; PWD=/home/bob ; USER=root ; C
OMMAND=/usr/bin/systemctl restart cron
2026-10-07T14:12:08.810383+00:00 tp-socle sudo:      bob : command not allowed ; TTY=pts/1 ; PWD=/ho
me/bob ; USER=root ; COMMAND=/usr/bin/systemctl stop cron
2026-10-07T14:12:08.869805+00:00 tp-socle sudo:      bob : command not allowed ; TTY=pts/1 ; PWD=/ho
me/bob ; USER=root ; COMMAND=/usr/bin/systemctl restart ssh
2026-10-07T14:12:08.926353+00:00 tp-socle sudo: pam_unix(sudo-i:session): session closed for user bo
b
2026-10-07T14:12:09.058039+00:00 tp-socle sudo:      etu : TTY=pts/0 ; PWD=/home/etu ; USER=root ; C
OMMAND=/usr/bin/grep sudo:.*bob /var/log/auth.log
etu@tp-socle:~$
```

Figure 27 : étape 6 : journal de sudo (journalctl indisponible, lecture de auth.log)

Ligne du journal correspondant à une commande refusée :

```
2026-10-07T14:12:08.810383+00:00 tp-socle sudo:      bob : command not
allowed ; TTY=pts/1 ;
PWD=/home/bob ; USER=root ; COMMAND=/usr/bin/systemctl stop cron
```

```
etu@tp-socle:~$ sudo EDITOR=nano visudo -f /etc/sudoers.d/tp-operateurs
/etc/sudoers.d/tp-operateurs:4:26: syntax error
bob ALL=(root /usr/bin/ls
                      ^
What now?
```

Figure 28 : étape 7 : visudo détecte l'erreur de syntaxe

```
etu@tp-socle:~$ sudo cat /etc/sudoers.d/tp-operateurs
# Seance 3 - bob peut redemarrer cron, et rien d'autre
bob ALL=(root) /usr/bin/systemctl restart cron

etu@tp-socle:~$ sudo visudo -c
/etc/sudoers: parsed OK
/etc/sudoers.d/README: parsed OK
/etc/sudoers.d/tp-operateurs: bad permissions, should be mode 0440
etu@tp-socle:~$
```

Figure 29 : après correction : fichier propre et vérification avec visudo -c

```
etu@tp-socle:~$ sudo mv /etc/sudoers.d/tp-operateurs /etc/sudoers.d/tp.operateurs
etu@tp-socle:~$ sudo -u bob sudo -l
[sudo] password for bob:
Sorry, user bob may not run sudo on tp-socle.
etu@tp-socle:~$ sudo mv /etc/sudoers.d/tp.operateurs /etc/sudoers.d/tp-operateurs
etu@tp-socle:~$ sudo -u bob sudo -l
[sudo] password for bob:
Matching Defaults entries for bob on tp-socle:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User bob may run the following commands on tp-socle:
    (root) /usr/bin/systemctl restart cron
etu@tp-socle:~$
```

Figure 30 : étape 8 : avec un point dans le nom, la règle ne s'applique plus

```
etu@tp-socle:~$ sudo cat /etc/sudoers.d/tp-operateurs
# Seance 3 - bob peut redemarrer cron, et rien d'autre
bob ALL=(root) /usr/bin/systemctl restart cron

bob ALL=(root) /usr/bin/less /srv/partage/confidentiel.txt

etu@tp-socle:~$
```

Figure 31 : étape 9 : règle less ajoutée par le collègue



Figure 32 : bob ouvre le fichier confidentiel avec sudo less

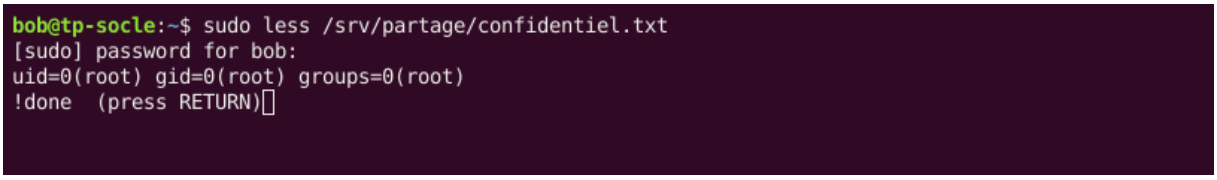


Figure 33 : !id depuis less : la commande tourne en root

Avec `!id` on voit `uid=0(root) gid=0(root) groups=0(root)` : less a lancé un shell en tant que root.

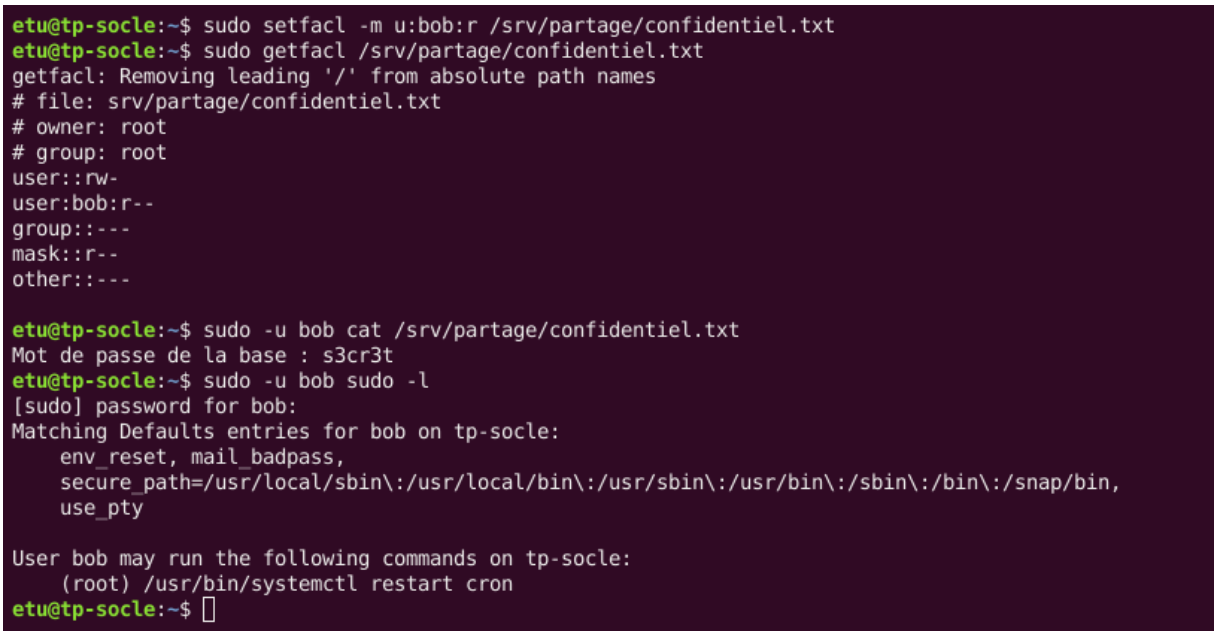


Figure 34 : étape 10 : règle less supprimée, lecture donnée par une ACL

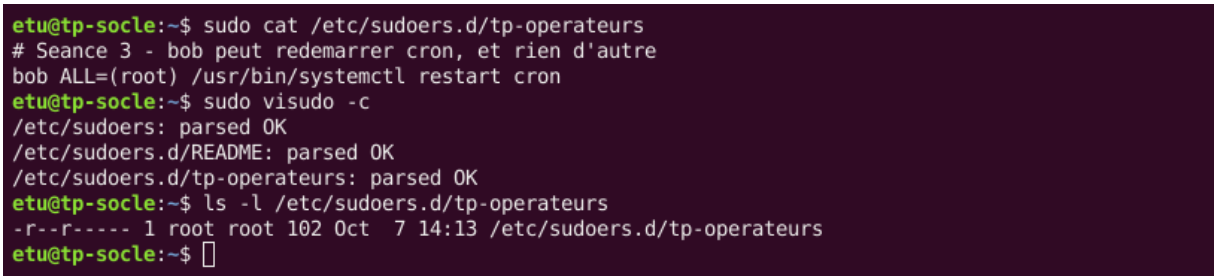


Figure 35 : version finale de /etc/sudoers.d/tp-operateurs, en 0440

Réaction de visudo à l'étape 7 :

visudo refuse d'installer le fichier : il affiche « /etc/sudoers.d/tp-operateurs:4:26: syntax error » avec la ligne fautive et un ^ sous la position de l'erreur, puis demande « What now? ». J'ai répondu e pour revenir dans l'éditeur et j'ai supprimé la ligne. La règle valide n'a jamais été cassée.

Réponse à l'étape 8 :

Non, avec le nom `tp.operateurs` bob n'a plus aucun droit (« Sorry, user bob may not run sudo »). La directive `@includedir /etc/sudoers.d` ignore les fichiers dont le nom contient un point ou finit par ~. Après avoir redonné le nom d'origine, la règle s'applique de nouveau.

Réponse à la question 10 :

Avec cette règle bob a en réalité obtenu un shell root : less permet de lancer une commande avec `!`, et cette commande s'exécute avec les droits de less, donc root. La délégation « lire un fichier » donne en fait tous les droits sur la machine. Solution sans sudo : une ACL de lecture pour bob sur le fichier (`sudo setfacl -m u:bob:r /srv/partage/confidentiel.txt`). bob a déjà le droit de traverser `/srv/partage` (ACL `u:bob:x` de l'exercice 2). J'ai appliqué l'ACL, supprimé la règle less, puis vérifié : bob lit le fichier avec un simple cat et `sudo -l` ne liste plus que la règle cron.

Exercice 6 - Se connecter avec une clé SSH

```
etu@tp-socle:~$ ssh alice@localhost
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:z5+kz2zLiqqwd+ut6vzI7k7AeBAZHDkka8s8Th3BmFA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Figure 36 : étape 1 : première connexion, ssh demande de confirmer l'empreinte

```
etu@tp-socle:~$ ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
256 SHA256:z5+kz2zLiqqwd+ut6vzI7k7AeBAZHDkka8s8Th3BmFA root@vm (ED25519)
etu@tp-socle:~$
```

Figure 37 : dans un second terminal : empreinte réelle de la clé du serveur

Les deux empreintes sont identiques : `SHA256:z5+kz2zLiqqwd+ut6vzI7k7AeBAZHDkka8s8Th3BmFA`. J'ai donc répondu yes.



Figure 38 : connexion par mot de passe puis contenu de known_hosts

Le fichier known_hosts contient les clés publiques du serveur (ed25519, rsa, ecdsa). Le nom de la machine est haché (`|1|...`) car Ubuntu active `HashKnownHosts`.

```
etu@tp-socle:~$ ssh-keygen -t ed25519 -C "etu@tp-seance3"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/etu/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/etu/.ssh/id_ed25519
Your public key has been saved in /home/etu/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:sLN7fvhIkTsFs4QTR4dBkhVuHieIsgoq4ZwVYG+Cmx0 etu@tp-seance3
The key's randomart image is:
+--[ED25519 256]--+
| o    o=B+.      |
|o o  ..B..       |
|..E+. = O .      |
| +o+.  B O       |
|= o.  o S .      |
|=oo    o +       |
|++    . +.       |
|.      ooo.      |
|      .ooo.      |
+----[SHA256]-----+
etu@tp-socle:~$ ls -ld ~/.ssh
drwx------ 2 etu etu 4096 Oct  7 14:16 /home/etu/.ssh
etu@tp-socle:~$ ls -l ~/.ssh
total 16
-rw------- 1 etu etu 464 Oct  7 14:16 id_ed25519
-rw-r--r-- 1 etu etu  96 Oct  7 14:16 id_ed25519.pub
-rw------- 1 etu etu 978 Oct  7 14:16 known_hosts
-rw-r--r-- 1 etu etu 142 Oct  7 14:16 known_hosts.old
etu@tp-socle:~$
```

Figure 39 : étape 2 : génération de la paire de clés et droits des fichiers

```
etu@tp-socle:~$ ssh-copy-id -i ~/.ssh/id_ed25519.pub alice@localhost
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/etu/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are alr
eady installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to inst
all the new keys
alice@localhost's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'alice@localhost'"
and check to make sure that only the key(s) you wanted were added.

etu@tp-socle:~$ sudo ls -la /home/alice/.ssh
[sudo] password for etu:
total 12
drwx------ 2 alice alice 4096 Oct  7 14:16 .
drwxr-x--- 4 alice alice 4096 Oct  7 14:16 ..
-rw------- 1 alice alice   96 Oct  7 14:16 authorized_keys
etu@tp-socle:~$ sudo cat /home/alice/.ssh/authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIMJ6uoTkN/aBTpVSic/kfigKwH+zebDTFtpXXoBHOHIs etu@tp-seance3
etu@tp-socle:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIMJ6uoTkN/aBTpVSic/kfigKwH+zebDTFtpXXoBHOHIs etu@tp-seance3
etu@tp-socle:~$
```

Figure 40 : étape 3 : dépôt de la clé publique chez alice

La ligne de `/home/alice/.ssh/authorized_keys` est exactement le contenu de mon `id_ed25519.pub`. ssh-copy-id a aussi créé `/home/alice/.ssh` en 700 et authorized_keys en 600.

```
etu@tp-socle:~$ ssh alice@localhost
Enter passphrase for key '/home/etu/.ssh/id_ed25519':
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.18.44-fc-v77 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Wed Oct  7 14:16:25 2026 from 127.0.0.1
alice@tp-socle:~$ whoami
alice
alice@tp-socle:~$ exit
logout
Connection to localhost closed.
etu@tp-socle:~$
```

Figure 41 : étape 4 : connexion par clé, la passphrase est demandée

Étape 4 : ssh ne demande plus le mot de passe d'alice mais la phrase de passe de ma clé privée (« Enter passphrase for key '/home/etu/.ssh/id_ed25519' »). Ce secret est à moi, pas à alice : il sert seulement à déchiffrer ma clé privée en local, il ne passe jamais sur le réseau.

```
etu@tp-socle:~$ eval "$(ssh-agent -s)"
Agent pid 3856
etu@tp-socle:~$ ssh-add
Enter passphrase for /home/etu/.ssh/id_ed25519:
Identity added: /home/etu/.ssh/id_ed25519 (etu@tp-seance3)
etu@tp-socle:~$ ssh-add -l
256 SHA256:sLN7fvhIkTsFs4QTR4dBkhVuHieIsgoq4ZwVYG+Cmx0 etu@tp-seance3 (ED25519)
etu@tp-socle:~$ ssh alice@localhost
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.18.44-fc-v77 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Wed Oct  7 14:16:28 2026 from 127.0.0.1
alice@tp-socle:~$ exit
logout
Connection to localhost closed.
etu@tp-socle:~$
```

Figure 42 : étape 5 : avec l'agent, plus aucune question

Étape 6 : droits trop ouverts.

```
etu@tp-socle:~$ sudo chmod g+w /home/alice/.ssh/authorized_keys
[sudo] password for etu:
etu@tp-socle:~$ sudo ls -l /home/alice/.ssh/authorized_keys
-rw--w---- 1 alice alice 96 Oct  7 14:16 /home/alice/.ssh/authorized_keys
etu@tp-socle:~$ ssh alice@localhost 'echo connexion acceptee pour $(whoami)'
connexion acceptee pour alice
etu@tp-socle:~$
```

Figure 43 : chmod g+w : la clé est encore acceptée

Ma prédiction était un refus, mais la connexion par clé a réussi. Sur Debian et Ubuntu, OpenSSH est modifié pour tolérer l'écriture par le groupe quand ce groupe est le groupe privé de l'utilisateur et ne contient personne d'autre (`getent group alice` montre que le groupe alice est vide). Pour reproduire vraiment le cas « un autre compte peut modifier le fichier », j'ai mis le fichier dans le groupe projet (où il y a carole) :

```
etu@tp-socle:~$ getent group alice
alice:x:30036:
etu@tp-socle:~$ sudo chgrp projet /home/alice/.ssh/authorized_keys
etu@tp-socle:~$ sudo ls -l /home/alice/.ssh/authorized_keys
-rw--w---- 1 alice projet 96 Oct  7 14:16 /home/alice/.ssh/authorized_keys
etu@tp-socle:~$ ssh alice@localhost
alice@localhost's password:

etu@tp-socle:~$
```

Figure 44 : fichier modifiable par le groupe projet : la clé est refusée, ssh demande un mot de passe

```
etu@tp-socle:~$ sudo grep sshd /var/log/auth.log | tail -n 3
2026-10-07T14:17:23.093789+00:00 tp-socle sshd[4030]: Authentication refused: bad ownership or modes
 for file /home/alice/.ssh/authorized_keys
2026-10-07T14:17:23.401371+00:00 tp-socle sshd[4030]: Connection closed by authenticating user alice
 127.0.0.1 port 55186 [preauth]
2026-10-07T14:17:23.460707+00:00 tp-socle sudo:      etu : TTY=pts/0 ; PWD=/home/etu ; USER=root ; C
OMMAND=/usr/bin/grep sshd /var/log/auth.log
etu@tp-socle:~$
```

Figure 45 : raison du refus dans le journal du serveur

```
etu@tp-socle:~$ sudo chgrp alice /home/alice/.ssh/authorized_keys
etu@tp-socle:~$ sudo chmod 600 /home/alice/.ssh/authorized_keys
etu@tp-socle:~$ ssh alice@localhost 'echo connexion OK en tant que $(whoami)'
connexion OK en tant que alice
etu@tp-socle:~$
```

Figure 46 : retour au groupe alice et aux droits 600 : la clé est de nouveau acceptée

```
etu@tp-socle:~$ cat > ~/.ssh/config <<'EOF'
> Host tp-alice
>     HostName localhost
>     User alice
>     IdentityFile ~/.ssh/id_ed25519
> EOF
etu@tp-socle:~$ cat ~/.ssh/config
Host tp-alice
    HostName localhost
    User alice
    IdentityFile ~/.ssh/id_ed25519
etu@tp-socle:~$ ssh tp-alice
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.18.44-fc-v77 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Wed Oct  7 14:16:29 2026 from 127.0.0.1
alice@tp-socle:~$ hostname; whoami
tp-socle
alice
alice@tp-socle:~$ exit
logout
Connection to localhost closed.
etu@tp-socle:~$
```

Figure 47 : étape 7 : raccourci tp-alice dans ~/.ssh/config

Fichier	Droits relevés
~/.ssh	700 (drwx------)
~/.ssh/id_ed25519	600 (-rw-------)
~/.ssh/id_ed25519.pub	644 (-rw-r--r--)
/home/alice/.ssh/authorized_keys	600 (-rw-------)

Empreinte affichée à la première connexion : ED25519
`SHA256:z5+kz2zLiqqwd+ut6vzI7k7AeBAZHDkka8s8Th3BmFA`

Message du serveur à l'étape 6 : « Authentication refused: bad ownership or modes for file /home/alice/.ssh/authorized_keys » (avec le fichier modifiable par le groupe projet ; avec le simple g+w sur le groupe privé alice, aucun refus, voir l'explication ci-dessus).

Réponse à la question 8 :

Non, la clé privée n'a jamais quitté `/home/etu/.ssh/id_ed25519`. Seule la clé publique a été copiée dans authorized_keys d'alice. À chaque connexion, le serveur envoie un défi et le client le signe avec la clé privée ; seule la signature est transmise, le serveur la vérifie avec la clé publique.

Exercice 6 bis - Authentification par clé depuis le poste (Plan B)

J'ai suivi le Plan B : le client est le compte `poste`, le serveur est mon compte `etu` sur la même machine (IP_VM = localhost).

```
poste@tp-socle:~$ ssh etu@localhost
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:z5+kz2zLiqqwd+ut6vzI7k7AeBAZHDkka8s8Th3BmFA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Figure 48 : empreinte affichée par ssh côté poste

```
etu@tp-socle:~$ ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
256 SHA256:z5+kz2zLiqqwd+ut6vzI7k7AeBAZHDkka8s8Th3BmFA root@vm (ED25519)
etu@tp-socle:~$
```

Figure 49 : empreinte réelle affichée dans la console de la machine de TP

Les empreintes sont identiques, j'ai répondu yes et donné le mot de passe d'etu.

Question : pourquoi la console du serveur est fiable ?

Avec deux machines distinctes, la console (écran de la VM ou accès physique) ne passe pas par le réseau. Si quelqu'un se place entre le poste et le serveur (homme du milieu), il peut présenter sa propre clé dans la connexion SSH, mais il ne peut pas modifier ce que la console du vrai serveur affiche. C'est donc une source indépendante pour comparer.

```
poste@tp-socle:~$ ssh-keygen -t ed25519 -C "poste@tp-seance3"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/poste/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/poste/.ssh/id_ed25519
Your public key has been saved in /home/poste/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:f0F0lDXdL56cwaPnwQVPhpE+KO6XsfKaepGe+n1l9t8 poste@tp-seance3
The key's randomart image is:
+--[ED25519 256]--+
|            ..+B+|
|           . .= *|
|            .+ =.|
|          ... B +|
|        S. o.= O |
|         .+ o.O +|
|         o.o.* =.|
|          *++ o o|
|        o=+=o.  E|
+----[SHA256]-----+
poste@tp-socle:~$
```

Figure 50 : étape 2 : paire de clés du compte poste

```
poste@tp-socle:~$ cat ~/.ssh/id_ed25519.pub | ssh etu@localhost "mkdir -p ~/.ssh && chmod 700 ~/.ssh
 && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
etu@localhost's password:
poste@tp-socle:~$ ssh etu@localhost
Enter passphrase for key '/home/poste/.ssh/id_ed25519':
```

Figure 51 : étapes 3 et 4 : le dépôt sans ssh-copy-id demande le mot de passe d'etu, la connexion suivante la passphrase

```
poste@tp-socle:~$ cat ~/.ssh/id_ed25519.pub | ssh etu@localhost "mkdir -p ~/.ssh && chmod 700 ~/.ssh
 && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
etu@localhost's password:
poste@tp-socle:~$ ssh etu@localhost
Enter passphrase for key '/home/poste/.ssh/id_ed25519':
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.18.44-fc-v77 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Wed Oct  7 14:17:26 2026 from 127.0.0.1
etu@tp-socle:~$ ls -la ~/.ssh
total 32
drwx------ 2 etu etu 4096 Oct  7 14:17 .
drwxr-x--- 4 etu etu 4096 Oct  7 14:17 ..
-rw------- 1 etu etu   98 Oct  7 14:17 authorized_keys
-rw-rw-r-- 1 etu etu   87 Oct  7 14:17 config
-rw------- 1 etu etu  464 Oct  7 14:16 id_ed25519
-rw-r--r-- 1 etu etu   96 Oct  7 14:16 id_ed25519.pub
-rw------- 1 etu etu  978 Oct  7 14:16 known_hosts
-rw-r--r-- 1 etu etu  142 Oct  7 14:16 known_hosts.old
etu@tp-socle:~$ exit
logout
Connection to localhost closed.
poste@tp-socle:~$
```

Figure 52 : étape 4 : connexion par clé sur le compte etu

```
poste@tp-socle:~$ cat > ~/.ssh/config <<'EOF'
> Host vm-tp
>     HostName localhost
>     User etu
>     IdentityFile ~/.ssh/id_ed25519
> EOF
poste@tp-socle:~$ eval "$(ssh-agent -s)"
Agent pid 4190
poste@tp-socle:~$ ssh-add
Enter passphrase for /home/poste/.ssh/id_ed25519:
Identity added: /home/poste/.ssh/id_ed25519 (poste@tp-seance3)
poste@tp-socle:~$ ssh vm-tp 'whoami; hostname'
etu
tp-socle
poste@tp-socle:~$
```

Figure 53 : étape 5 : raccourci vm-tp

```
poste@tp-socle:~$ echo "test" > un_fichier.txt
poste@tp-socle:~$ scp un_fichier.txt vm-tp:/tmp/
un_fichier.txt                                100%    5    11.7KB/s   00:00
poste@tp-socle:~$ scp vm-tp:/etc/os-release .
os-release                                    100%  400   262.5KB/s   00:00
poste@tp-socle:~$ ls -l un_fichier.txt os-release; ls -l /tmp/un_fichier.txt
-rw-r--r-- 1 poste poste 400 Oct  7 14:17 os-release
-rw-rw-r-- 1 poste poste   5 Oct  7 14:17 un_fichier.txt
-rw-rw-r-- 1 etu etu 5 Oct  7 14:17 /tmp/un_fichier.txt
poste@tp-socle:~$
```

Figure 54 : étape 6 : copie de fichiers dans les deux sens avec scp

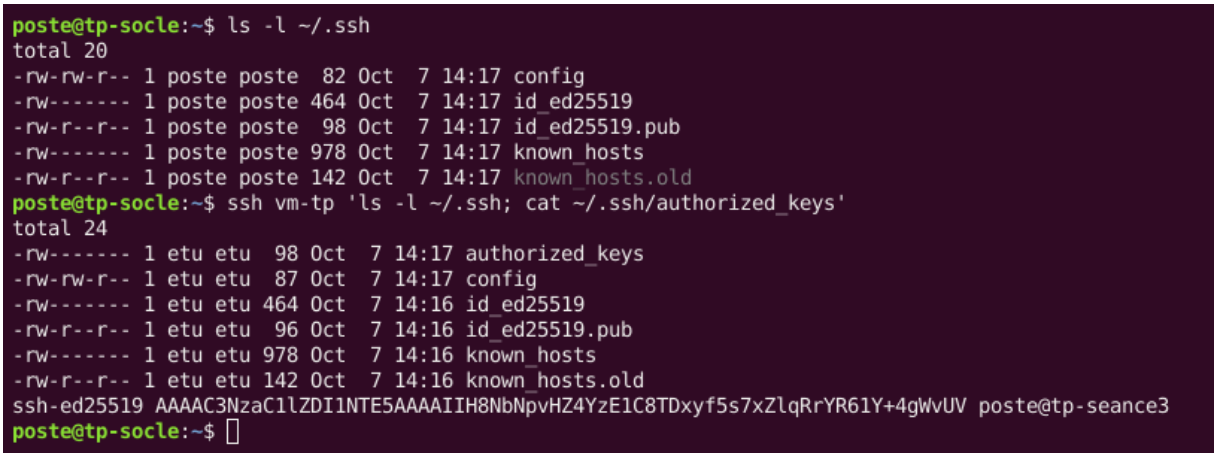


Figure 55 : question 7 : où se trouvent les clés

Vérification	**Résultat**
Empreinte affichée par ssh sur le poste	SHA256:z5+kz2zLiqqwd+ut6vzI7k7AeBAZHDkka8s8Th3BmFA
Empreinte affichée dans la console de la machine de TP	SHA256:z5+kz2zLiqqwd+ut6vzI7k7AeBAZHDkka8s8Th3BmFA (identique)
Secret demandé à l'étape 4	la passphrase de la clé du compte poste (plus le mot de passe d'etu)
`ssh vm-tp` fonctionne	oui (whoami renvoie etu)

Explication de la commande de l'étape 3 :

`cat ~/.ssh/id_ed25519.pub` écrit la clé publique sur la sortie standard. Le pipe | l'envoie sur l'entrée standard de `ssh etu@localhost`, qui la transmet à la commande lancée sur le serveur. Côté serveur : `mkdir -p ~/.ssh` crée le dossier s'il n'existe pas, `chmod 700` le protège, `cat >> ~/.ssh/authorized_keys` ajoute ce qui arrive sur l'entrée standard à la fin du fichier (sans écraser les clés déjà présentes), puis `chmod 600` protège le fichier. Les && font qu'une étape ne s'exécute que si la précédente a réussi. La notion de la séance 1 utilisée est la redirection : le pipe et l'entrée standard (stdin), ainsi que >>.

Réponse à la question 7 :

La clé privée est uniquement dans `/home/poste/.ssh/id_ed25519` (sur le poste). La clé publique est dans `/home/poste/.ssh/id_ed25519.pub` et sa copie dans `/home/etu/.ssh/authorized_keys` (sur le serveur). Si le poste est volé, il faut se connecter au serveur par un autre moyen et supprimer la ligne `poste@tp-seance3` de authorized_keys, puis générer une nouvelle paire de clés sur un poste sain. La passphrase laisse un peu de temps mais ne remplace pas la révocation.

Exercice 7 - Durcir le service SSH

Avant de commencer j'ai vérifié que `ssh vm-tp` fonctionne par clé depuis le compte poste (exercice 6 bis), et j'ai gardé une session ouverte pendant les modifications.

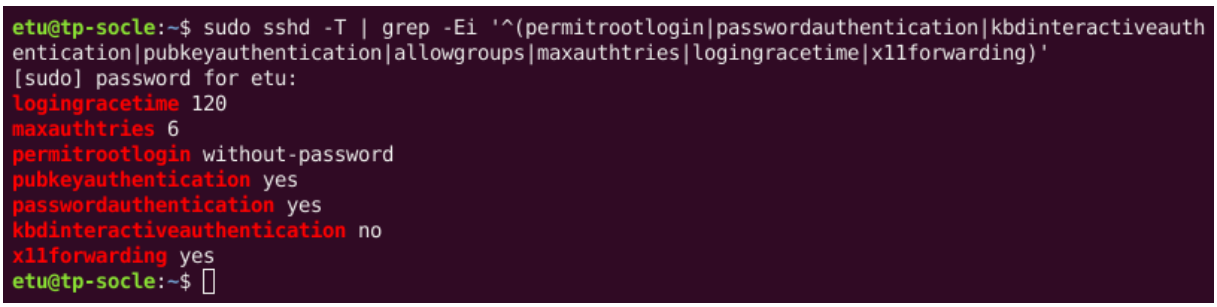


Figure 56 : étape 1 : valeurs avant durcissement

Avant : `permitrootlogin without-password`, `passwordauthentication yes`, `maxauthtries 6`, `logingracetime 120`, `x11forwarding yes`, et pas de AllowGroups.

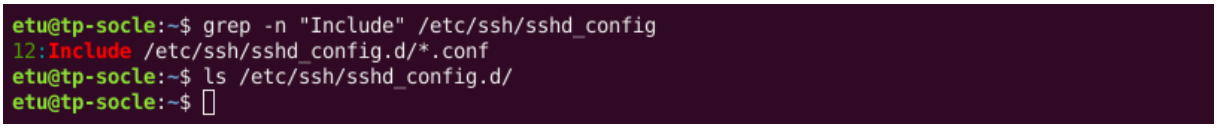


Figure 57 : étape 2 : ligne Include et contenu de sshd_config.d

La ligne `Include /etc/ssh/sshd_config.d/*.conf` est la ligne 12, donc au tout début du fichier principal : les fichiers du dossier sont lus avant le reste de sshd_config. Le dossier était vide.



Figure 58 : étape 3 : création de 10-tp-durcissement.conf et test de syntaxe

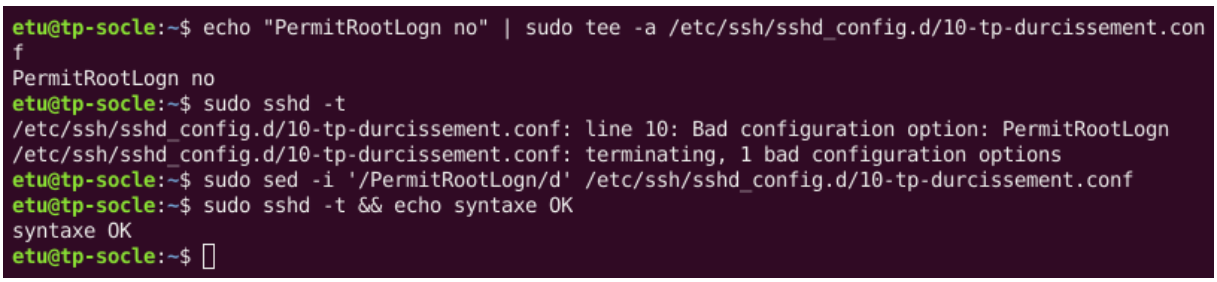


Figure 59 : étape 4 : faute de frappe détectée par sshd -t

Message de `sshd -t` à l'étape 4 :

```
/etc/ssh/sshd_config.d/10-tp-durcissement.conf: line 10: Bad configuration
option: PermitRootLogn
/etc/ssh/sshd_config.d/10-tp-durcissement.conf: terminating, 1 bad
configuration options
```

```
etu@tp-socle:~$ sudo sshd -T | grep -Ei '^(permitrootlogin|passwordauthentication|kbdinteractiveauth
entication|pubkeyauthentication|allowgroups|maxauthtries|logingracetime|x11forwarding)'
logingracetime 30
maxauthtries 3
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
kbdinteractiveauthentication no
x11forwarding no
allowgroups projet
allowgroups sudo
etu@tp-socle:~$
```

Figure 60 : étape 5 : valeurs appliquées après durcissement

Étape 5 : oui, toutes les valeurs ont changé. AllowGroups apparaît sur deux lignes (une par groupe).

```
etu@tp-socle:~$ sudo systemctl reload ssh
System has not been booted with systemd as init system (PID 1). Can't operate.
Failed to connect to bus: Host is down
etu@tp-socle:~$ sudo service ssh reload
 * Reloading OpenBSD Secure Shell server's configuration sshd                    [ OK ]
etu@tp-socle:~$
```

Figure 61 : étape 6 : rechargement de la configuration (service, faute de systemd)

```
etu@tp-socle:~$ ssh alice@localhost 'echo connecte en tant que $(whoami)'
Enter passphrase for key '/home/etu/.ssh/id_ed25519':
connecte en tant que alice
etu@tp-socle:~$ ssh -o PubkeyAuthentication=no alice@localhost
alice@localhost: Permission denied (publickey).
etu@tp-socle:~$ ssh bob@localhost
bob@localhost: Permission denied (publickey).
etu@tp-socle:~$ ssh root@localhost
root@localhost: Permission denied (publickey).
etu@tp-socle:~$
```

Figure 62 : étape 7 : tests depuis un nouveau terminal

```
poste@tp-socle:~$ ssh vm-tp 'echo connecte en tant que $(whoami)'
Enter passphrase for key '/home/poste/.ssh/id_ed25519':
connecte en tant que etu
poste@tp-socle:~$ ssh -o PubkeyAuthentication=no etu@localhost
etu@localhost: Permission denied (publickey).
poste@tp-socle:~$
```

Figure 63 : étape 7 : tests depuis le compte poste

Connexion	**Prédiction**	**Résultat**
alice par clé	réussit	réussit (passphrase demandée, « connecte en tant que alice »)
alice sans clé	échoue	échoue : Permission denied (publickey)
bob	échoue	échoue : Permission denied (publickey), bloqué par AllowGroups
root	échoue	échoue : Permission denied (publickey)
etu depuis le poste, par clé	réussit	réussit (« connecte en tant que etu »)
etu depuis le poste, sans clé	échoue	échoue : Permission denied (publickey)



Figure 64 : étape 8 : raison du refus de bob dans le journal

Raison du refus de bob dans le journal : « User bob from 127.0.0.1 not allowed because none of user's groups are listed in AllowGroups ». bob n'est ni dans projet ni dans sudo. Même chose pour root.

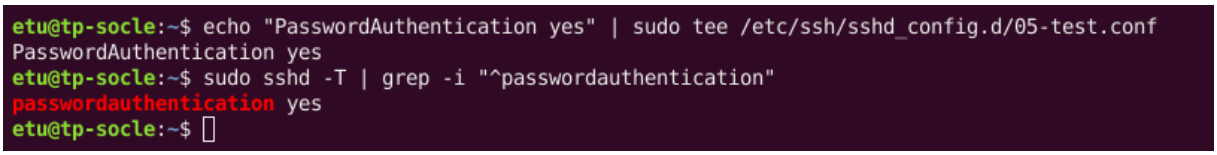


Figure 65 : étape 9 : 05-test.conf l'emporte sur le fichier de durcissement



Figure 66 : 90-test.conf est ignoré, puis suppression du fichier de test

Fichier de test	Valeur appliquée
05-test.conf	passwordauthentication yes
90-test.conf	passwordauthentication no

Contenu de 10-tp-durcissement.conf :

```
# Seance 3 - durcissement du serveur SSH
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
AllowGroups projet sudo
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
```

Réponse à la question 10 :

Pour sshd, la première valeur lue l'emporte, et les fichiers sont lus dans l'ordre alphabétique. `50-cloud-init.conf` est lu avant `60-durcissement.conf`, donc `PasswordAuthentication yes` reste appliqué et mon durcissement ne sert à rien pour cette directive, sans aucun message d'erreur. Il faut donner au fichier un numéro plus petit (10-...) ou corriger le fichier cloud-init, et toujours vérifier avec `sshd -T`.

Exercice 8 - Synthèse : qui peut faire quoi ?

Les prédictions ont été écrites avant les tests, à partir de la configuration mise en place. Plan B : « depuis le poste » signifie depuis le compte poste.

```
etu@tp-socle:~$ sudo -u bob cat /srv/partage/rapport.txt
cat: /srv/partage/rapport.txt: Permission denied
etu@tp-socle:~$ sudo -u carole sh -c 'echo ligne carole >> /srv/partage/rapport.txt' && echo ecritur
e OK
ecriture OK
etu@tp-socle:~$ sudo -u bob ls /srv/partage
ls: cannot open directory '/srv/partage': Permission denied
etu@tp-socle:~$ sudo -u carole sh -c 'echo ligne carole >> /srv/partage/commun/nouveau.txt' && echo
ecriture OK
ecriture OK
etu@tp-socle:~$ sudo -u bob cat /srv/partage/confidentiel.txt
Mot de passe de la base : s3cr3t
etu@tp-socle:~$ sudo -l -U bob | tail -n 2
User bob may run the following commands on tp-socle:
    (root) /usr/bin/systemctl restart cron
etu@tp-socle:~$
```

Figure 67 : vérification manuelle des droits fichiers et sudo

```
etu@tp-socle:~$ ssh tp-alice 'whoami'
Enter passphrase for key '/home/etu/.ssh/id_ed25519':
alice
etu@tp-socle:~$ ssh bob@localhost
bob@localhost: Permission denied (publickey).
etu@tp-socle:~$
```

Figure 68 : vérification des connexions SSH d'alice et de bob

```
poste@tp-socle:~$ ssh vm-tp 'whoami'
Enter passphrase for key '/home/poste/.ssh/id_ed25519':
etu
poste@tp-socle:~$ ssh -o PubkeyAuthentication=no etu@localhost
etu@localhost: Permission denied (publickey).
poste@tp-socle:~$
```

Figure 69 : vérification des connexions SSH depuis le poste

Action	Compte	Prédiction	Vérifié	Commande utilisée
Lire rapport.txt	bob	non	non	`sudo -u bob cat /srv/partage/rapport.txt`
Modifier rapport.txt	carole	oui	oui	`sudo -u carole sh -c 'echo ... >> rapport.txt'`
Lister /srv/partage	bob	non	non	`sudo -u bob ls /srv/partage`
Modifier commun/nouveau.txt	carole	oui	oui	`sudo -u carole sh -c 'echo ... >> nouveau.txt'`
Lire confidentiel.txt	bob	oui	oui	`sudo -u bob cat /srv/partage/confidentiel.txt`
Redémarrer cron	bob	oui	oui	`sudo -l -U bob`
Redémarrer ssh	bob	non	non	`sudo -l -U bob`
Se connecter en SSH	alice (par clé)	oui	oui	`ssh tp-alice 'whoami'`
Se connecter en SSH	bob	non	non	`ssh`

				`bob@localhost`
SSH depuis le poste	etu (par clé)	oui	oui	`ssh vm-tp 'whoami'`
SSH depuis le poste	etu (mot de passe)	non	non	`ssh -o PubkeyAuthentication=no etu@localhost`

Ce que j'avais oublié :

Dans ce tableau final toutes mes prédictions étaient justes. Pendant le TP je m'étais trompé trois fois : j'avais oublié que mon compte etu n'est pas dans le groupe projet (ls refusé à l'exercice 1), que visudo crée le fichier en 640 alors que sudo attend 440, et que le patch Debian d'OpenSSH accepte un authorized_keys en g+w quand le groupe est le groupe privé de l'utilisateur (exercice 6).

Étape 4 : automatisation avec verif_tp.sh. J'ai ajouté des tests sur le listage, l'héritage, la lecture de confidentiel.txt, les règles sudo (`sudo -l -U bob`), l'ACL par défaut (`getfacl`) et trois directives SSH (`sshd -T`).

```
etu@tp-socle:~$ ls -l ~/verif_tp.sh
-rwx------ 1 etu etu 2648 Oct  7 14:18 /home/etu/verif_tp.sh
etu@tp-socle:~$ sudo bash ~/verif_tp.sh ; echo "code : $?"
[sudo] password for etu:
[OK] bob ne peut pas lire rapport.txt
[OK] carole peut modifier rapport.txt
[OK] bob ne peut pas lister /srv/partage
[OK] carole peut modifier commun/nouveau.txt
[OK] bob peut lire confidentiel.txt
[OK] ACL par defaut group:projet presente sur commun
[OK] bob peut redemarrer cron avec sudo
[OK] bob n'a pas d'autre droit sudo (pas de ssh, pas de less)
[OK] sshd refuse les mots de passe
[OK] sshd refuse root
[OK] AllowGroups projet sudo
--- 11 reussi(s), 0 echec(s)
code : 0
etu@tp-socle:~$
```

Figure 70 : verif_tp.sh avant l'erreur volontaire : 11 tests réussis, code 0

```
etu@tp-socle:~$ sudo chmod 640 /srv/partage/rapport.txt
etu@tp-socle:~$ sudo bash ~/verif_tp.sh ; echo "code : $?"
[OK] bob ne peut pas lire rapport.txt
[KO] carole ne peut pas modifier rapport.txt (attendu : autorise)
[OK] bob ne peut pas lister /srv/partage
[OK] carole peut modifier commun/nouveau.txt
[OK] bob peut lire confidentiel.txt
[OK] ACL par defaut group:projet presente sur commun
[OK] bob peut redemarrer cron avec sudo
[OK] bob n'a pas d'autre droit sudo (pas de ssh, pas de less)
[OK] sshd refuse les mots de passe
[OK] sshd refuse root
[OK] AllowGroups projet sudo
--- 10 reussi(s), 1 echec(s)
code : 1
etu@tp-socle:~$
```

Figure 71 : après chmod 640 : le script détecte le problème, code 1

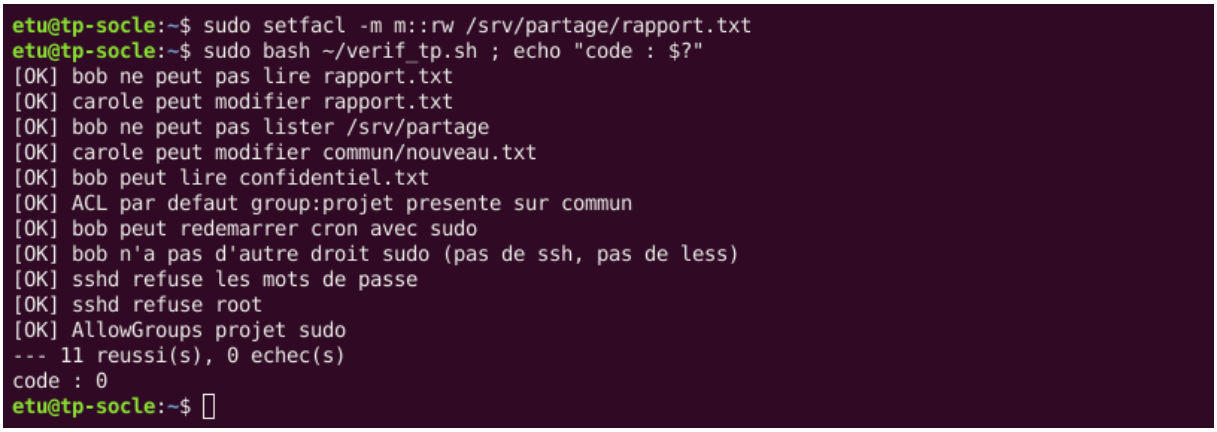


Figure 72 : après setfacl -m m::rw : tout est de nouveau [OK]

Le script détecte bien l'erreur : après `chmod 640`, le masque repasse à r-- et la ligne de carole devient [KO], le code de retour passe à 1. Après `setfacl -m m::rw` tous les tests sont de nouveau [OK] (code 0).

Réponse à la question de l'étape 4 :

`test -r` lancé avec l'identité de bob demande directement au noyau si bob a le droit, en tenant compte de tout : ses groupes, les ACL nommées, le masque, et le droit de traverser chaque répertoire du chemin. `ls -l` ne montre que les bits rwx d'un seul fichier (et le masque à la place du groupe quand il y a un +), sans les entrées ACL ni les droits des dossiers parents. On peut donc mal interpréter `ls -l`, alors que le test donne le résultat réel.

Fichiers à rendre

/etc/sudoers.d/tp-operateurs (version finale, droits 0440)

```
# Seance 3 - bob peut redemarrer cron, et rien d'autre
bob ALL=(root) /usr/bin/systemctl restart cron
```

/etc/ssh/sshd_config.d/10-tp-durcissement.conf

```
# Seance 3 - durcissement du serveur SSH
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
AllowGroups projet sudo
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
```

Script verif_tp.sh complété

```
#!/bin/bash
# verif_tp.sh - verifie automatiquement le tableau de l'exercice 8
# Usage : sudo bash verif_tp.sh
REUSSIS=0
ECHECS=0

ok() { echo "[OK] $1"; REUSSIS=$(( REUSSIS + 1 )); }
ko() { echo "[KO] $1"; ECHECS=$(( ECHECS + 1 )); }

# Un acces qui doit etre REFUSE
if sudo -u bob test -r /srv/partage/rapport.txt; then
```

```
  ko "bob peut lire rapport.txt (attendu : refuse)"
else
  ok "bob ne peut pas lire rapport.txt"
fi

# Un acces qui doit etre AUTORISE
if sudo -u carole test -w /srv/partage/rapport.txt; then
  ok "carole peut modifier rapport.txt"
else
  ko "carole ne peut pas modifier rapport.txt (attendu : autorise)"
fi

# bob ne doit pas pouvoir lister /srv/partage (il a seulement x)
if sudo -u bob ls /srv/partage > /dev/null 2>&1; then
  ko "bob peut lister /srv/partage (attendu : refuse)"
else
  ok "bob ne peut pas lister /srv/partage"
fi

# carole doit pouvoir modifier commun/nouveau.txt (ACL par defaut du groupe
projet)
if sudo -u carole test -w /srv/partage/commun/nouveau.txt; then
  ok "carole peut modifier commun/nouveau.txt"
else
  ko "carole ne peut pas modifier commun/nouveau.txt (attendu : autorise)"
fi

# bob lit confidentiel.txt grace a l'ACL (et plus avec sudo less)
if sudo -u bob test -r /srv/partage/confidentiel.txt; then
  ok "bob peut lire confidentiel.txt"
else
  ko "bob ne peut pas lire confidentiel.txt (attendu : autorise)"
fi

# ACL : l'entree par defaut du groupe projet doit etre sur commun
if getfacl -p /srv/partage/commun 2>/dev/null | grep -q
"^default:group:projet:rwx"; then
  ok "ACL par defaut group:projet presente sur commun"
else
  ko "ACL par defaut group:projet absente sur commun"
fi

# Regle sudo : bob peut redemarrer cron ...
if sudo -l -U bob | grep -q "/usr/bin/systemctl restart cron"; then
  ok "bob peut redemarrer cron avec sudo"
else
  ko "bob ne peut pas redemarrer cron avec sudo (attendu : autorise)"
fi

# ... mais rien d'autre (ni ssh, ni less)
if sudo -l -U bob | grep -Eq "restart ssh|less|\(ALL"; then
  ko "bob a plus de droits sudo que prevu"
else
  ok "bob n'a pas d'autre droit sudo (pas de ssh, pas de less)"
fi

# SSH : mot de passe interdit, root interdit, groupes autorises
```

```
if sshd -T | grep -qx "passwordauthentication no"; then
  ok "sshd refuse les mots de passe"
else
  ko "sshd accepte encore les mots de passe"
fi

if sshd -T | grep -qx "permitrootlogin no"; then
  ok "sshd refuse root"
else
  ko "sshd accepte root"
fi

if sshd -T | grep -qx "allowgroups projet" && sshd -T | grep -qx "allowgroups
sudo"; then
  ok "AllowGroups projet sudo"
else
  ko "AllowGroups mal configure"
fi

echo "--- $REUSSIS reussi(s), $ECHECS echec(s)"
[ "$ECHECS" -eq 0 ]
```